

Name: - SHUBHAM DAS

Date: - 20/09/2026

S.A.P I. D: - 590033500

Subject: - Programming In C

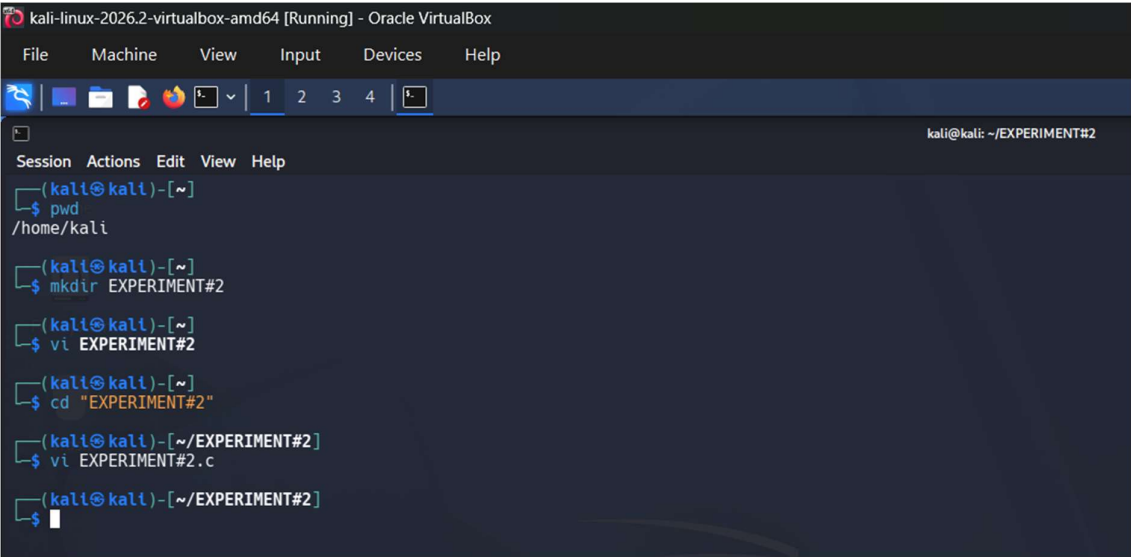
Experiment#2: -

Purpose of the experiment: - Creation of a small C program from the source code to a working executable programme and observation of the files created during each stage of compilation.

Environment: -

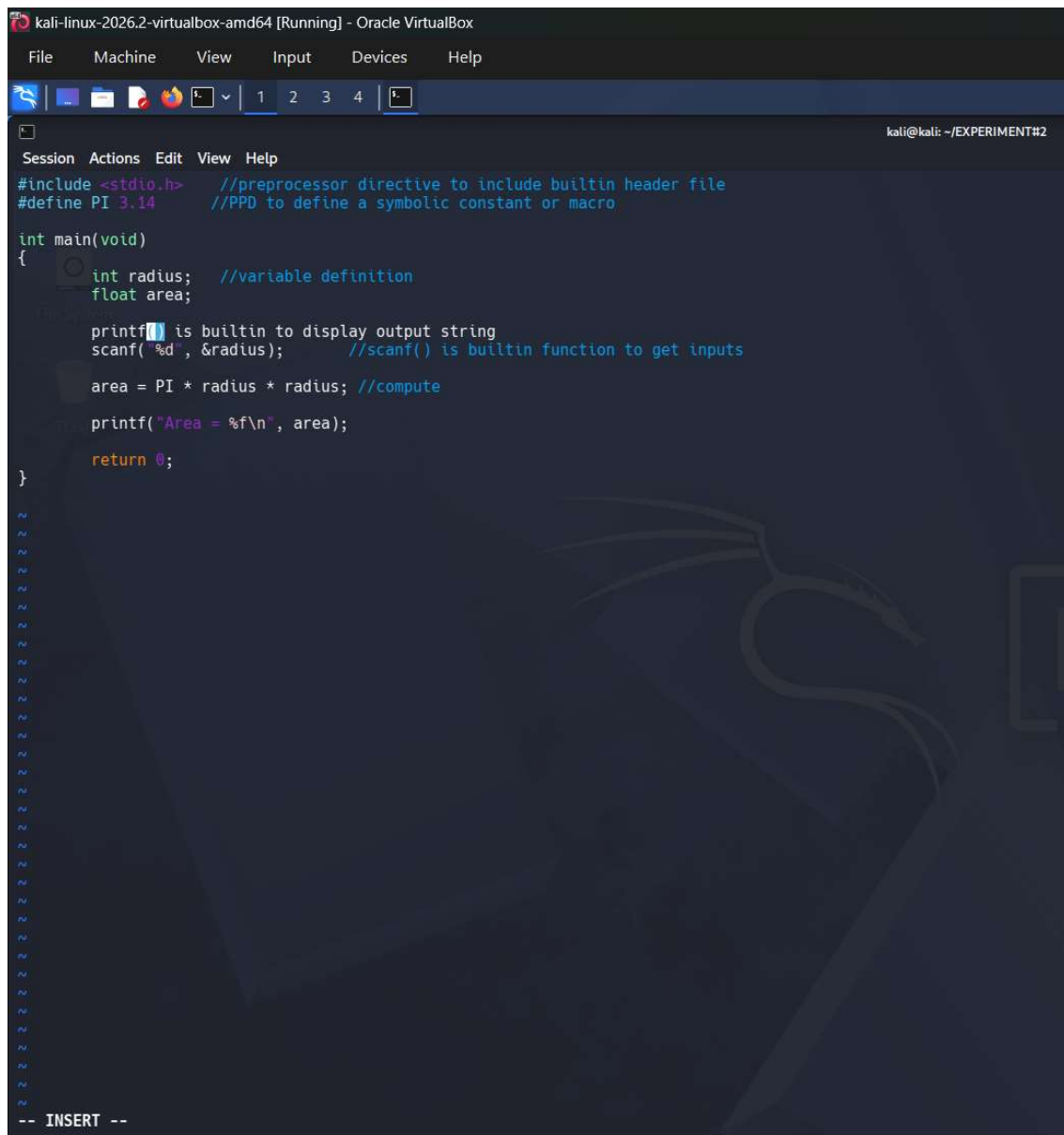
- Kali Linux VM
- Oracle VirtualBox
- C Programming Language

Aim of the experiment: - To write a C program in Kali Linux VM, save and compile it, observe the four compilation stages and all the intermediate files, execute the final program, and then verify the final result.



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/EXPERIMENT#2
Session Actions Edit View Help
(kali@kali)-[~]
$ pwd
/home/kali
(kali@kali)-[~]
$ mkdir EXPERIMENT#2
(kali@kali)-[~]
$ vi EXPERIMENT#2
(kali@kali)-[~]
$ cd "EXPERIMENT#2"
(kali@kali)-[~/EXPERIMENT#2]
$ vi EXPERIMENT#2.c
(kali@kali)-[~/EXPERIMENT#2]
$
```



```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/EXPERIMENT#2
Session Actions Edit View Help
#include <stdio.h> //preprocessor directive to include builtin header file
#define PI 3.14 //PPD to define a symbolic constant or macro

int main(void)
{
    int radius; //variable definition
    float area;

    printf is builtin to display output string
    scanf( "%d", &radius); //scanf() is builtin function to get inputs

    area = PI * radius * radius; //compute

    printf("Area = %f\n", area);

    return 0;
}

-- INSERT --
```

Inner Workings of the C Programme: -

1. '#include<stdio.h>' = Makes the standard input & output functions, 'printf()' and 'scanf()', available
2. '#define P.I 3.14' = Creates the mathematical constant P.I, which is commonly used in most arithmetic problems, such as calculation of area.
3. 'Int radius' = Stores the radius in whole number integers entered by the user.

4. 'Float area' = Stores the calculated area which may or may not contains the numeric values in decimal number system
5. 'scanf("%d", &radius)' = Reads the input of radius from the keyboard and stores it in the radius
6. 'area = P.I * radius * radius' = Applies the formula for the area of a circle
7. 'Print ("Area=%f\n", area)' = Display of the final output that is the calculation of the total area. '%f' prints the six digits after the decimal by default

Creation and operation of the desired C programme: -

- Step 1: - Open Oracle VirtualBox. Start Kali Linux VM. Launch the terminal command. Check current directory with 'pwd'. Create folder. List directory Open-source file with 'vi'.
- Step2: - Open the new file in Kali Linux. Use the Insert Mode so that the C programme can be typed in an editor. The code will include the header file, P.I, variable declarations, input, calculation, output and return statement.
- Step3: - Enter the source code. Enter the complete area of the circle programme. Include comments beside the important lines to describe the purpose of the preprocessor directives, variables, input, and computation.
- Step 4: - Save the file and exit the editor. Type the programme so that Kali Linux will be switched from Insert Mode to Command Mode. The file will be written to disk and the editor will be closed by using the write-and-quit command. In :wq, w means write (save) and q means quit. This is required saving step before the final compilation.
- Step 5: - Compile and execute the programme. The C compile driver that was included, invoked cc. Because no output filename was supplied, the default executable was named a.out. The executable will be then run from the current directory using ./a.out. A radius of any value can be entered as input now.
- Step 6: - Preserve the intermediate compilation file. The -save temps option will be used so that the normally temporary pre-processed assembly, and the object files will be used for further inspection. This command will then be made use for further verification of the files. The saved files provide direct evidence of pre-processing, compilation, assembly, and linking stages.

```
(kali㉿kali)-[~/EXPERIMENT#2]
$ vi "EXPERIMENT#2.c"

(kali㉿kali)-[~/EXPERIMENT#2]
$ cc "EXPERIMENT#2.c"

(kali㉿kali)-[~/EXPERIMENT#2]
$ ./a.out
ENTER THE RADIUS: █
```

```
(kali㉿kali)-[~/EXPERIMENT#2]
$ vi "EXPERIMENT#2.c"

(kali㉿kali)-[~/EXPERIMENT#2]
$ cc "EXPERIMENT#2.c"

(kali㉿kali)-[~/EXPERIMENT#2]
$ ./a.out
ENTER THE RADIUS: 6
Area = 113.040001

(kali㉿kali)-[~/EXPERIMENT#2]
$ █
```

```
(kali㉿kali)-[~/EXPERIMENT#2]
$ vi "EXPERIMENT#2.c"

(kali㉿kali)-[~/EXPERIMENT#2]
$ cc "EXPERIMENT#2.c"

(kali㉿kali)-[~/EXPERIMENT#2]
$ ./a.out
ENTER THE RADIUS: 6
Area = 113.040001

(kali㉿kali)-[~/EXPERIMENT#2]
$ cc -save-temps "EXPERIMENT#2.c"

(kali㉿kali)-[~/EXPERIMENT#2]
$ ls █
```

The Four Stages of Proper Compilation of Any Programme in C Language

A single cc command performs several operations behind the scenes. The '-save-temps' option makes the intermediate results visible.

- Stage 1= Pre-processing: - Processing the '#include' and '#define' directives, removal of the comments and expansion of the macros. The pre-processor handles the lines in the beginning with a '#' before the C language compiler analyses the programme. In this particular experiment, it brings in declarations from 'stdio.h' and replaces every use of P.I with 3.14. Then the final conclusion of this experiment is the saved '.i' file.
- Stage 2= Compilation: - Checking and translation of the pre-processed C language programme into an assembly language. The compiler checks the

pre-processor programme and converts it into assembly instructions for the target system. This is where syntax and type-related errors are usually reported. The saved output is the '.s' file.

- Stage 3= Assembly: - Conversion of assembly instruction into re-locatable machine-code object data. The assembler translates the assembly instructions into machine-code object data. The object file is not yet a complete running programme because any kind of references to external library functions still needs to be resolved.
- Stage 4= Linkage: - Combination of the object file with required library codes such as 'printf ()' and 'scanf ()'. The linker combines the object file with the required startup and standard library code. It resolves references to the functions such as 'printf ()' and 'scanf ()', and then produces the final executable programme. With no '-o' option, the default executable is 'a.out'

```
(kali㉿kali)-[~/EXPERIMENT#2]
$ vi "EXPERIMENT#2.c"

(kali㉿kali)-[~/EXPERIMENT#2]
$ cc "EXPERIMENT#2.c"

(kali㉿kali)-[~/EXPERIMENT#2]
$ ./a.out
ENTER THE RADIUS: 6
Area = 113.040001

(kali㉿kali)-[~/EXPERIMENT#2]
$ cc -save-temps "EXPERIMENT#2.c"

(kali㉿kali)-[~/EXPERIMENT#2]
$ ls
a-EXPERIMENT#2.i  a-EXPERIMENT#2.o  a-EXPERIMENT#2.s  a.out  EXPERIMENT#2.c

(kali㉿kali)-[~/EXPERIMENT#2]
$
```

Final Result: - The experiment is completed successfully. The C language programme was created and saved in Vim, compiled with the cc command and executed as './a.out'. The cc '-save-temps' also preserved the '.i', '.s', and '.o' files, confirming the pre-processing, compilation, assembly and the linking stage.

Final Conclusion: - This experiment correctly demonstrated to us the complete journey of compilation and execution of C language programme in a very practical manner. We initially started off with only a readable '.c' file, saved it correctly, and used the compiler to transform it through four stages in a running executive programme. Observing the '.i', '.s', and '.o' files, made the normally hidden compilation process easier to understand. The final output confirmed both the

programme logic and the build process worked correctly in Kali Linux VM operating on Oracle VirtualBox.